SDC HACKATHON 2026

Phase 2: Final Project & Prototype Report

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 1 November 2026

Instructions: Complete every field below and submit as a single PDF document named "SDC2026_TeamName_Final.pdf" together with your pitch deck and a working deployment link. Answers must comply with the word limits stated for each open-ended question. Section 3 requires concrete, verifiable evidence. Unsupported claims will not be credited by judges.

1. Team & Project Overview

Team Name	UTeMku Cemerlang
Project / Product Name	PaySah
Final Live Deployment URL	https://paysah.replit.app
Codebase Repository URL	https://github.com/UTeMkuCemerlang/PaySah_Final
Name (Leader)	Chong Kai Li
Email & phone (Leader)	kailichong148@gmail.com | 0146312883

2. Project Description

[Maximum 250 words]

PaySah helps Malaysian micro and small merchants review payment screenshots against orders and independent bank evidence. It addresses repetitive receipt collection and checking, and the risk of treating a scheduled, incomplete or reused screenshot as proof of payment.

Customers submit synthetic receipts through an order-linked Google Form or Telegram; merchants can also upload directly. AI extracts amount, recipient account, time, reference and transfer status. Server rules compare the fields with orders and synthetic bank credits, detect duplicates and present clear exception reasons. The Owner reviews the evidence, confirms payment and separately releases fulfilment. By default PaySah warns and blocks. The Owner may choose 'Approve anyway' at their own responsibility, with a reason and confirmation prompt. Every override is logged as MANUALLY_CONFIRMED ('Accepted by Owner'), never counted as Paid, and allocates no bank credit.

The Finalist version adds shop-specific, versioned policies, a Laundry demonstration, incremental activity history, draft-preserving refresh, bounded Telegram demo-link intake and a proposed Orbyte adapter. React/TypeScript, Express, PostgreSQL, private Replit storage, RunPod vision extraction and CPU OCR form the implementation.

The intended SDG contribution is better SME productivity (8.2), supporting enterprise growth (8.3). The prototype uses synthetic business data and has no live bank or payment gateway connection. Local tests and synthetic diagnostics demonstrate technical behaviour; merchant time savings and real-world accuracy remain to be measured.

3. Upgraded Product & Technical Execution

3.1 Final System Architecture

Provide a detailed description (and, if possible, an attached technical diagram) of your fully built system. Explain how your application integrates GenAI APIs, database structures, and backend/frontend frameworks into one unified solution.

Dashboard, Telegram and Google Forms/Apps Script feed the Express API. Order-linked intake validates the request and stores receipt images in private Replit App Storage; PostgreSQL holds tenant-scoped orders, receipts, extraction jobs, verification results, policy versions and activity events. A persistent worker claims database jobs using a 180-second lease. RunPod Serverless/vLLM serves Qwen2.5-VL-7B-Instruct for structured extraction. Tesseract.js supplies CPU reading and fallback, with provider provenance recorded. The live advisory third reader, gpt-5.4-mini via KrackedDevs (litellm.rapidscreen.io), receives receipt images only when GPU and CPU disagree. It cannot change fields, rules or payment, and the workflow continues when it is disabled.
Deterministic checks compare fields with order amounts and independent synthetic bank credits. Hash duplicate detection, transaction-allocation controls, permission checks, tenant filtering and version checks protect updates. PaySah warns and blocks by default. The Owner may override at their own responsibility using 'Approve anyway', with a reason and confirmation prompt. Each override is logged as MANUALLY_CONFIRMED ('Accepted by Owner'), never counted as Paid. Confirmation and fulfilment release are separate API actions. The React interface refreshes workspace data every five seconds while retaining drafts.
System flow: customer/UI -> authenticated intake/API -> private image store + PostgreSQL job -> worker -> model/CPU -> verification -> Owner decision -> separate release -> activity history. The Orbyte adapter is a local team-defined mock, not an official or live integration. The team-built mock payment page (/pay) and bank feed are simulated; no real bank, gateway or payment is involved.

3.2 Implementation & Hackathon Deliverables

Describe what was successfully built, integrated, and deployed during the 48-hour physical sprint. Highlight any specific problem-solving steps your team took to pivot or improve the project from the Phase 1 preliminary design.

[Maximum 300 words]

Relative to Phase 1, the Finalist source implements versioned shop policies, a Laundry SME scenario, denied-action auditing, incremental activity, five-second refresh with draft retention, bounded Telegram demo-link intake and build-commit metadata. The interface focuses on the Owner; legacy Cashier permissions remain in the API. Partial payments can be supplemented or rejected. Default warnings/blocks permit an Owner override with responsibility, a reason and confirmation prompt; it is logged as MANUALLY_CONFIRMED ('Accepted by Owner'), never counted as Paid.

A team-built mock payment page (/pay) and the bank feed are simulated; no real bank, gateway or payment is involved. KrackedDevs gpt-5.4-mini provides live advice on GPU/CPU disagreements without changing fields, rules or payment.

CPU parser improvements raised correct fields on 36 diagnostic images from 190/252 to 224/252; on 81 new-value images using the same layouts, from 416/567 to 483/567. Rotation and low resolution still block valid receipts.

Published 1 November local validation records 179 passing tests, zero failures/skips, a successful build and 5/5 simulated intake checks at application commit 966a19b. The 31 October structured fixture run recorded 8/8 expected decisions and zero false admissions, wrong releases or false blocks; this did not measure vision or real payments.

On 1 November, the public Replit health endpoint returned FINALIST, RunPod mode and commit aa203ec00f4b3f1927ced7de67e41022b5617441. Database readiness was not checked; health alone does not establish workflow success or parity with tested commit 966a19b. Real Telegram/Form tests on 24 October used CPU fallback after GPU timeout.

Development history is retained; these records do not establish that every change occurred within the official 48-hour physical sprint.

4. Environmental & SDG Impact Verification

Validate the real-world impact of your built solution. Show concrete data that validate your environmental or social value proposition, compared against the baseline and KPI defined in previous phase submission.

*The data and evidence you provide in this section will be used to assess **SDG Impact, Technical Execution and Innovation scores.** Vague claims or missing verification will weaken your score across all three categories.*

Metric Category	Your Response
Target SDG Metric Achievement *Direct comparison between your Phase 1 baseline and your Phase 2 finalized prototype numbers* *e.g., "Achieved a 24.5% reduction in computational execution overhead against the 20% initial target."*	Primary SDG 8.2; supporting 8.3. Phase 1 baseline: 2 merchant file actions per receipt (download + upload); target: 0 on connected channels and at least 30% lower median active handling time. Connected intake is implemented, but observed employee file-action counts and paired human timing have not been collected. Neither target is claimed as achieved. No carbon, energy or national SDG-indicator savings are claimed.
Verification Evidence Method *How was this data verified?* *e.g., codebase algorithmic profiling logs, hardware power-meter simulation readings, or database transaction counters.*	Published local evidence: 179/179 tests and 5/5 simulated intake checks on 1 November; 8/8 structured fixture decisions on 31 October. On the same 36 synthetic images, historical Qwen2.5-VL GPU extraction scored 248/252 fields (98.41%), latest KrackedDevs gpt-5.4-mini 249/252 (98.81%), and CPU OCR 224/252 (88.89%). Runs used different dates, prompt/parser versions and conditions; these are not a ranking of models. Latest gpt-5.4-mini exact receipts: 33/36; CPU: 27/36. Model rule checks produced 6/36 false-eligible cases, CPU 0/36; no approval/release was executed. Correlated, non-held-out images do not establish real-bank accuracy. Human-impact protocol: docs/FINALIST_MEASUREMENT.md.
Socio-Economic / Civic Value *How does your deployed application translate into a scalable, real-world solution capable of being supported by civic or corporate partners?*	Automatic collection and explainable review could reduce repetitive administration and improve evidence traceability for small merchants. A merchant association, university or industry partner could recruit consenting SME pilots and evaluate actual task time, correction burden and errors. Benefits remain a testable operational hypothesis.

Metric Category	Your Response
Deployment & Commercial Feasibility *e.g., modeled as an API service plugin designed for easy integration into existing corporate or institutional systems.*	A hosted per-shop subscription is the proposed model; no price or measured cost per receipt is set. Replit hosts UI/API/worker, database and private images; RunPod supplies inference. A team-defined Orbyte mock illustrates an integration path. Authorized bank ingestion, production authentication, security/privacy review and merchant validation are required before real financial use.

4.1 Mathematical Proof / Code Logic Snippet (Optional, but holds bonus marks)

Paste the specific code section, mathematical formula, simulation log output, or calculation logic that proves the impact claims made above.

This proof is graded across three categories at once: ***Impact and Feasibility*** *(does the math support your claim),* ***Technical Execution*** *(does it reflect what you actually built), as well as* ***Innovation and Creativity*** *(is the approach original).] Do not leave this section generic or copied from Phase 1.*

Phase 1 action target: (2 - 0) / 2 x 100 = 100% proposed reduction. This is workflow arithmetic, not observed achievement.
Human time reduction = (median manual active seconds - median PaySah active seconds) / median manual active seconds x 100. No paired observations exist, so no reduction percentage is reported.
Synthetic extraction on the same 36 images: historical Qwen2.5-VL GPU 248 / 252 x 100 = 98.41%; latest KrackedDevs gpt-5.4-mini 249 / 252 x 100 = 98.81%; CPU OCR 224 / 252 x 100 = 88.89%. Dates, prompt/parser versions and conditions differ; no model ranking is claimed. Latest exact receipts: gpt-5.4-mini 33/36; CPU 27/36. The gpt-5.4-mini rule run had 6/36 false-eligible cases; no approval or release was executed.
Published local fixture output (31 October, source 1531bb6): decisions 8/8; false admissions 0; wrong releases 0; false blocks 0. Provider: FIXTURE. This small structured test proves only those expected API/database outcomes.
Reproduce locally from src/ with a dedicated test database: npm run demo:gate; npm run eval:e2e. Human observations are scored using npm run score:finalist -- <observations.json>. Retain unsuccessful attempts and unmatched pairs; do not substitute API latency for staff handling time.

5. Market Viability & Future Scalability Plan

SDC aims to encourage solutions that transition from innovations into market-ready applications. If given access to corporate funding, industry grants, or partnership opportunities (e.g., through SDC's Rotary network or joint research initiatives at XMUM), how would your team scale this prototype to process real-world data at volume?

[Maximum 250 words]

Start with a small, consented pilot among merchants handling transfer screenshots. Measure paired manual/PaySah tasks, merchant file actions, decision accuracy, unresolved cases and total elapsed time. Include normal, high-value, partial, duplicate, wrong-account and missing-evidence cases. Use results to decide whether the proposed per-shop subscription creates enough value; no validated price or savings claim is available yet.

Funding would prioritize authorized bank-statement ingestion, secure identity and tenant onboarding, recovery/backup procedures, privacy notices, retention/deletion controls and independent security review. Orbyte integration would require an agreed official API, service authentication, tenant mapping and durable retry checkpoints; the existing mock is only a local demonstration.

The current architecture has one API and one extraction worker, with leased database jobs and per-tenant serialized writes. Additional workers, GPU autoscaling and concurrent-shop load tests are planned. Throughput, maximum shop count and production latency are unmeasured. Track inference spend, hosting/database/storage costs and completed receipts over the same period before reporting cost per receipt.

University and SME partners could support usability trials and data governance; industry partners could provide authorized transaction evidence. Broader deployment depends on measured reliability and an explicit policy for human overrides. AI remains an extraction aid, while bank evidence and recorded human decisions govern payment status.

6. Final Submission Checklist

Tick all boxes that apply before submitting.

- [x] The application is fully compiled, live, and accessible via the deployment link provided in Section 1.
- [x] The GitHub (or equivalent) repository contains a clean, well-documented README with installation instructions.
- [x] The GitHub (or equivalent) repository shows the history of incremental code updates made throughout the official live hacking window.
- [x] The final pitch deck has been exported and saved as a flattened PDF file.
- [x] All fields in this template have been completed and word limits respected.

7. IP & Originality Declaration

7.1 Pre-Existing / Prior-Owned Work

Does this submission include any code, design, research, or other work that any team member created or owned before the start of this hackathon?

☐ Yes ☑ No ☑ No change from Phase 1 declaration

If yes, briefly describe what was pre-existing and which team member(s) owned it:

7.2 Third-Party Code, Datasets, or Models

Does your project use any third-party code libraries, open-source frameworks, public/private datasets, or pre-trained AI models that your team did not create?

☑ Yes ☐ No ☐ No change from Phase 1 declaration

If yes, list them below (open-source use is permitted provided it is declared and properly licensed/attributed):

Name	Type (code/dataset/model)	License
React/React DOM 19.2.0; Vite 7.3.7; Express 5.1.0; Multer 2.4.0; Zod 4.1.12; pg 8.16.3; Three.js 0.186.1	Code	MIT
TypeScript 5.9.3; Sharp 0.35.5; Tesseract.js 7.0.0; Playwright 1.64.1; vLLM	Code	Apache-2.0
Node.js 24; @replit/object-storage 1.0.0; @tesseract.js-data/eng 1.0.0; tsx; concurrently	Runtime / code / OCR data	MIT; third-party notices apply
PostgreSQL 17; embedded-postgres 17.10.0-beta.17	Database / local helper	PostgreSQL License / MIT
Qwen/Qwen2.5-VL-7B-Instruct	Pre-trained model	Apache-2.0

Name	Type (code/dataset/model)	License
gpt-5.4-mini via KrackedDevs (litellm.rapidscreen.io): live advisory third reader and offline diagnostic	Hosted model/API	Provider terms; not redistributed
Team-created receipts, orders and bank credits	Synthetic dataset	Team-created; repository Apache-2.0

7.3 Employment-Related IP Conflicts

Is any team member currently bound by an employment, internship, or consulting agreement under which their employer could claim ownership of work they create during this hackathon?

☐ Yes ☑ No ☑ No change from Phase 1 declaration

If yes, please describe the nature of the agreement and which member it concerns:

8. Declaration

By submitting this report, the team leader confirms that the deployed application and data presented above are the original work of the team, completed within the official SDC 2026 hackathon period.



[Signature]

1/11/2026

[Date]